Virtueller Speicher

Wie viele Einträge hat eine Seitentabelle erster Stufe?

Anzahl Einträge einer Seitentabelle = $2^{PT1\ Größe}$

Wie groß (in Byte) ist eine Seite, wie groß ist eine Kachel?

Seitengröße = Kachelgröße = 2^{Offset}

Wie viele Seiten belegt das Stack-Segment?

Seitenzahl Stacksegment = $\frac{\text{Größe in Byte}}{2^{Offset}}$

Wie viele Seitentabellen 2. und 3. Stufe werden für das Text- und Heap- Segment benötigt?

Seitenzahl Textsegment u. Heapsegment = $\frac{\text{Größe in Byte}}{2^{Offset}}$

Seitentabellen 3.Stufe = $\frac{\text{Seitenzahl Textsegment u. Heapsegment}}{2^{PT3}}$

Seitentabellen 2.Stufe = $\frac{\text{Seitentabellen 3.Stufe}}{2^{PT2}}$

Seitentabellen 1.Stufe = $\frac{\text{Seitentabellen 2.Stufe}}{2^{PT1}}$

Wie viele Kacheln verwaltet das Betriebssystem?

Anzahl Kacheln = $\frac{\text{Größe Hauptspeicher + Größe Swap}}{\text{Kachelgröße}}$ **(z.B. 4K Seiten (4096))**

Auf welche Größe kann der physische Speicher maximal ausgebaut werden? (Hauptspeicher plus Swap-Space)

Adressbreite in Bit (PT1+PT2+…+Offset) z.B. 28 Bit → 2^{28} **Byte (Wörter) adressierbar (256MB).**
Der virtuelle Speicher kann also auf 256MB ausgebaut werden.

Benötigt das System weitere Seitentabellen zweiter Stufe für das Stacksegment?

Seitentabellenzahl Stack größer oder kleiner 2^{Offset} ?

Kleiner → es wird keiner benötigt

Größer → Es ist eine weitere Seitentabelle 2. Stufe notwendig da der Stack in einem anderen Adressbereich liegt und sich daher keine Seitentabelle mit dem HuS Segment teilen kann

Allgemein

Exponent = $\frac{\ln 512}{\ln 2}$ **= 9 --> 2^9**

Größe Seitentabelle = $\textbf{Anzahl adressierbarer Seiten} \times \textbf{Größe eines Eintrags}$

(z.B. 8K * 3Byte =24KB)

Anzahl Adressen erreichen/ansprechen einer Seitentabelle = $2^{PT\ Größe} \times 2^{Offset}$

Seitenzahl Stacksegment = $\frac{\text{Größe in Byte}}{2^{Offset}}$

Seitenzahl Textsegment u. Heapsegment = $\frac{\text{Größe in Byte}}{2^{Offset}}$

Seitentabellenzahl Textsegment u. Heapsegment = $\frac{\text{Seitenzahl}}{Einträge}$

Seitentabellenzahl Stacksegment = $\frac{\text{Seitenzahl}}{Einträge}$

virt. Adresse				phys. Adresse
1	0	2	491	425 451
0	18	244	418	X
62	31	254	81	146 001
1	3	0	375	169 847
1	0	1	125	366 717
Y				114 000
62	31	255	263	495 367

X: Für die Adresse in PT1[0] (d. h. für 1193) ist keine PT2 angegeben.
Y: Für die Kachel-Nr. 222 gibt es keinen Eintrag in einer PT3.

Synchronisation

Nur unsynchronisierte Semaphoren in Quellcode

Deadlock:
Es kann in einen Deadlock geraten; Es gibt mindestens einen Zustand aus dem keine Kante herausführt

Kein Deadlock:
Es kann nicht in einen Deadlock geraten; aus jedem Zustand gibt es einen Übergang in einen anderen Zustand

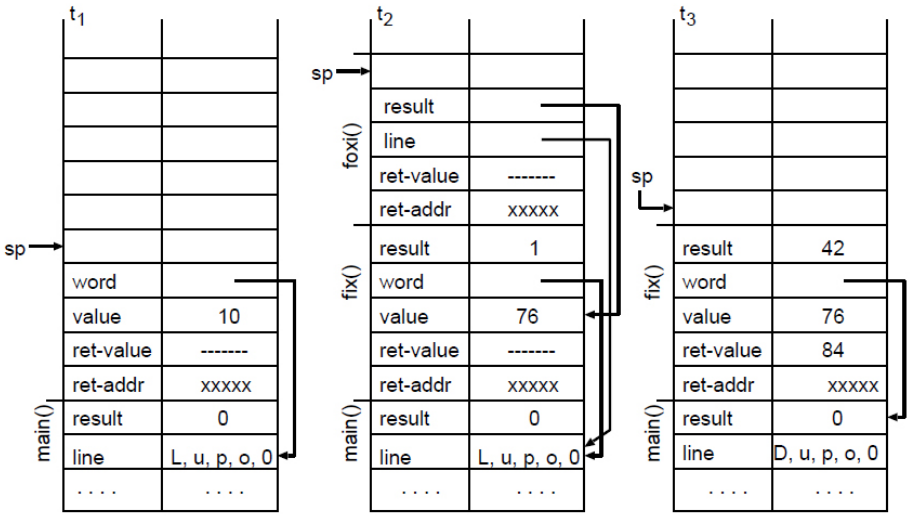
```
Prozess P_AB{
 while(true){
     S1.down()
     A();
     S3.up()
     S3.down();
     C();
     S1.up();
   }
}
```

Scheduling

Wann wird der Scheduler aktiv ? → (blockieren, deblockieren, Zeitscheibe)

Welcher Prozess war bisher schon rechnend **(kommt auf Scheduler Art an!)**

Funktionsaufruf



Ersetzungsstrategien

Optimale Reihenfolge

- Freie Plätze nach Reihenfolge auffüllen
- Ersetzt wird die am weitest entfernte Seite
- Wenn diese Seite schon eingelagert ist passiert nix

Clock Algorithmus

- Freie Plätze nach Reihenfolge auffüllen und Pointer auf nächste Kachel verschieben
- Wenn ein Platz zum Einlagern benötigt wird
    - Pointer läuft los setzt jedes (1) zu (0) runter
    - Beim nächsten Feld mit (0) (frei zum Überschreiben) wird die Seite eingelagert und der Pointer auf die nächste Kachel verschoben
- Wenn eine Seite schon eingelagert ist wird der Wert von (0) auf (1) gesetzt
    - Bei (1) bleibt (1)
    - Pointer bleibt stehen

NRU Algorithmus

1. Bit steht für Referenz R
2. Bit steht für Modify M

ZS = R Bit bei allen auf 0 setzen (evtl. auch beide !!! s.Text)

4/r = R Bit auf 1

2/w = M Bit und R Bit auf 1

Seite mit geringster Bit-Zahl wird ersetzt (Bei gleicher Bit-Zahl ???)

Bei eingetragener 4(1,0) und Einlagerung 4/r → keine Änderung

LRU Algorithmus

Es wird die Seite ausgelagert die schon am längsten nicht angesprochen wurde

Aging

- ZS = Zähler wird Halbiert → 5 wird zu 2 bei 3-Bit 101 → 010 (von Links eine Null auffüllen)
- Die Seite mit dem niedrigsten Zähler wird ersetzt
- Beim Aufruf schon eingetragener Seite wird Höchstwertiges Bit auf 1 gesetzt
- Bei neu eingelagerter Seite nur Höchstwertiges Bit auf 1 setzten

2-Bit	
00	0
01	1
10	2
11	3

3-Bit	
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

4-Bit	
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

Dateisystem mit I-Nodes

Wie Groß kann eine Datei in diesem Dateisystem maximal sein?

1. Zeiger pro Block = $\frac{\textit{Blockgröße}}{\textit{Bytes pro Zeiger}}$
2. Anzahl Blöcke: z.B. **1+1+256+256²** = **65795 Blöcke**
3. **Anzahl Blöcke** $\times$ **Blockgröße** → **269496320Byte**

Wie groß kann das Dateisystem maximal sein (Begründung)?

z.B. **16 Byte pro Zeiger (16 Byte * 8)** → Es können max. 2^{128} **Blöcke** adressiert werden
2^{140} **Byte** = $2^{128}\textit{Blöcke} \times 2^{12}$ **Blockgröße (Byte pro Block)**

Wie viele Blöcke belegt eine Datei, die 140 MB Daten enthält.

z.B. 140 MB → $\frac{140 \times 2^{20}}{\text{Blockgröße (Byte pro Block)}}$ = **Blöcke Für Daten**

Blöcke Für Daten – Direkte Blöcke (z.B. 1) – Zeiger pro Block = Restblöcke
(über zweifach indirekten Block)

Einfach indirekte Blöcke $= \frac{\text{Restblöcke}}{\text{Zeiger pro Block}}$

Blöcke Für Daten + Einfach indirekte Blöcke + Anzahl der einzelnen 1 und 2 fach indirekten Blöcke

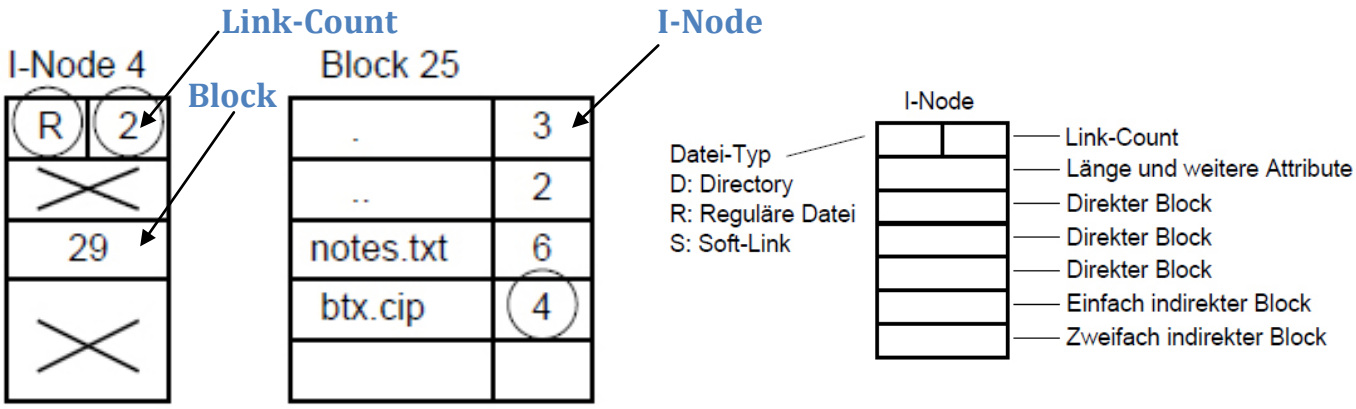
D = Directory (Verzeichnis)
R = Reguläre Datei
S = Soft-Link

Softlink → **Verzeichnis** (Inhalt des Blocks = kompletter Pfad /etc/prog.conf)
benötigt I-Node und Block mit Inhalt + Eintrag im Verzeichnis
Hardlink → **Gleicher I-Node Eintrag**
benötigt nur eigenen I-Node + Eintrag ins Verzeichnis

Link-Count: Anzahl der Hard-Links auf diesen I-Node (**.** und **..** werden nicht mitgezählt)

. = (Zeiger auf sich selbst) Nr. des I-Nodes der für diesen Block zuständig ist

.. = (ein Ordner höher in der Hierarchie) Nr. des I-Nodes der für den Parent-Directory zuständig ist



Was wird angelegt, welche Werte werden wo eingetragen bzw. verändert?

Neu: Ein Block (z.B. 16) für die zusätzlichen Daten und ein indirekter Block (z.B 15), der einen Verweis auf den Datenblock (in diesem Fall 16) enthält.

Änderungen: Im I-Node 5 Verweis auf indirekten Block eintragen (in diesem Fall 15), Länge der Datei anpassen (+4096). In der Block-Bitmap die beiden neu belegten Blöcke als belegt markieren (in diesem Fall Stelle 15 und 16 auf 1 setzen).

Eine Partition der Größe **2GB** soll durch ein FAT-Datei-System verwaltet werden. Die Größe eines Blocks beträgt **4KB**, ein Zeiger auf einen Block besteht aus **4 Byte**.

Wie groß ist die FAT für dieses Dateisystem?

$$\frac{\textbf{Partitionsgröße}}{\textbf{Blockgröße (Byte pro Block)}} = \textbf{Anzahl der Blöcke}$$

$$\textbf{Anzahl der Blöcke} \times \boldsymbol{Zeigergr}\text{öß}\boldsymbol{e} = \boldsymbol{FAT\ Gr}\text{öß}\boldsymbol{e}$$

FAT 32: hat Zeigergröße 4 Byte

FAT 24: hat Zeigergröße 3 Byte

FAT 16: hat Zeigergröße 2 Byte